TUGAS PRAKTIK PEMROGRAMAN BERBASIS OBJEK
MODUL 10: EXCEPTION HANDLING DAN VALIDASI STATE OBJEK

Nama Mahasiswa	: M. Davana Yudista Firdausy	**Mata Kuliah**	: Praktikum Pemrograman Berorientasi Objek
NRP	: 3125522002	**Pertemuan / Topik**	: 10 / Exception Handling & Validasi State
Kelas	: A D3 TI-SM	**Model Belajar**	: Project Based Learning (PBL)
Institusi	: PENS PSDKU Sumenep	**Proyek Domain**	: FleetSumenep - Rental Car Fleet & Bay Monitoring
GitHub Repository	: https://github.com/DavaYudist/PEMROGRAMAN-BERORIENTASI-OBJEK.git		

BAGIAN A: ANALISIS PERMASALAHAN & IDENTIFIKASI KONDISI KESALAHAN

Dalam perancangan sistem perangkat lunak berorientasi objek kelas industri (seperti sistem pemantauan armada rental mobil *FleetSumenep*), integritas dan keutuhan data (*state consistency*) adalah prasyarat mutlak. Prinsip fundamental Modul 10 menegaskan bahwa:

> **"Validasi wajib diselesaikan sebelum melakukan mutasi state; operasi yang ditolak akibat error atau kegagalan validasi wajib mempertahankan state awal objek secara utuh (Atomic & State Invariant)."**

Pada arsitektur sistem rental mobil FleetSumenep, telah diidentifikasi minimal empat kondisi kegagalan spesifik yang dapat mengancam konsistensi data jika tidak ditangani dengan mekanisme penanganan eksepsi yang ketat:

1. **Upaya Penyewaan Unit yang Sedang Beroperasi di Jalan (KendaraanTidakTersediaException):**
   Kondisi di mana pengguna mencoba menyewakan armada yang field `disewa == true` (status `DISEWA`). Jika mutasi langsung dijalankan tanpa validasi pra-syarat, nama penyewa aktif sebelumnya akan tertimpa (terkorupsi), mengakibatkan *double-booking* dan anomali billing. Sistem mencegah ini dengan melempar custom checked exception `KendaraanTidakTersediaException`.
2. **Upaya Penyewaan Unit dalam Kondisi Perbaikan (KendaraanRusakException):**
   Kondisi di mana kendaraan memiliki status `kondisiMesin.equalsIgnoreCase("Dalam Perbaikan")`. Dari aspek operasional keselamatan dan SLA perusahaan, armada yang mengalami kerusakan mekanik atau dalam pemeliharaan bengkel dilarang keras beroperasi. Sistem memvalidasi kelayakan fisik armada dan melempar checked exception `KendaraanRusakException` sebelum parameter sewa lainnya diproses.
3. **Validasi Input Parameter Transaksi & Nilai Atribut (IllegalArgumentException):**
   Kondisi di mana pemanggil metode menyuplai argumen tidak logis di luar batas domain, seperti durasi sewa ≤ 0 jam, persentase diskon di luar rentang [0.0% – 100.0%], plat nomor kosong/null, tarif sewa harian ≤ 0, atau odometer negatif. Sistem menerapkan *defensive input validation* pada tingkat konstruktor dan method dengan melempar `IllegalArgumentException` deskriptif.
4. **Pelanggaran Batas Kapasitas Bay Parkir Depot Pool (KapasitasPenuhException):**
   Kondisi saat administrator mencoba meregistrasikan armada baru namun kapasitas fisik bay parkir telah mencapai batas maksimum (32 bay), atau saat pengurangan kapasitas slot parkir (misal -4 bay) mencoba memotong bay parkir yang saat ini masih ditempati oleh unit armada aktif. Sistem memblokir aksi dan melempar `KapasitasPenuhException` guna menjaga konsistensi alokasi fisik lot parkir.

Rasionalitas Pemilihan Custom Domain-Specific Exception vs Generic Exception

Penggunaan custom exception bertipe checked (seperti `KendaraanTidakTersediaException` dan `KendaraanRusakException`) jauh lebih unggul dibandingkan melempar generic exception (seperti `Exception` atau `RuntimeException`) karena:

- **Ekspresivitas Semantik Domain:** Nama class eksepsi secara eksplisit mendeskripsikan konteks domain bisnis kegagalan, memudahkan debugging, penelusuran audit log, dan perancangan pesan kesalahan UI.
- **Enkapsulasi Data Kontekstual:** Custom exception membawa payload atribut khusus (misal `platNomor`, `kondisiMesin`, dan `kapasitasDiminta`) yang dapat diakses langsung oleh handler eksternal.
- **Penegakan Kontrak Compile-Time (Checked Exception):** Melalui klausa `throws` pada deklarasi method, compiler Java memaksa setiap layer pemanggil (desktop UI controller, REST bridge) untuk menyediakan penanganan multi-catch yang terstruktur, mengeliminasi risiko kelalaian developer saat mengintegrasikan modul.

BAGIAN B: IMPLEMENTASI KODE PROGRAM (CLASS & EXCEPTION ARCHITECTURE)

Berikut adalah implementasi arsitektur Exception Handling dan Defensive Programming yang telah dibangun pada modul `SourceCode/model/exception/` dan `SourceCode/model/`:

1. Implementasi Custom Checked Exception

```
package model.exception;

/**
 * Custom Checked Exception: KendaraanTidakTersediaException
 * Dilempar ketika armada berstatus DISEWA diminta kembali oleh pelanggan baru.
 */
public class KendaraanTidakTersediaException extends Exception {
    private String platNomor;

    public KendaraanTidakTersediaException(String message) { super(message); }
    public KendaraanTidakTersediaException(String message, String platNomor) {
        super(message);
        this.platNomor = platNomor;
    }
    public String getPlatNomor() { return platNomor; }
}

// KendaraanRusakException dilempar bila k.getKondisiMesin().equalsIgnoreCase("Dalam Perbaikan")
public class KendaraanRusakException extends Exception {
    private String platNomor, kondisiMesin;
    public KendaraanRusakException(String message, String platNomor, String kondisiMesin) {
        super(message);
        this.platNomor = platNomor;
        this.kondisiMesin = kondisiMesin;
    }
    public String getPlatNomor() { return platNomor; }
    public String getKondisiMesin() { return kondisiMesin; }
}
```

2. Defensive Programming & Deklarasi throws pada Method prosesSewa()

```
public synchronized String prosesSewa(String platNomor, String namaPenyewa, int durasiJam,
                                      boolean denganSupir, double diskonPersen)
        throws KendaraanTidakTersediaException, KendaraanRusakException, IllegalArgumentException {

    // 1. Validasi Eksistensi Armada
    Kendaraan k = findKendaraan(platNomor);
    if (k == null) throw new IllegalArgumentException("Kendaraan plat " + platNomor + " tidak ditemukan.");

    // 2. Validasi Kelayakan Operasional (Sebelum Mutasi State)
    if ("Dalam Perbaikan".equalsIgnoreCase(k.getKondisiMesin())) {
        throw new KendaraanRusakException("Unit sedang dalam perbaikan dan tidak layak operasi.",
                                          k.getPlatNomor(), k.getKondisiMesin());
    }

    // 3. Validasi Ketersediaan Unit (Cek apakah sedang disewa)
    if (k.isDisewa()) {
        throw new KendaraanTidakTersediaException("Kendaraan sedang disewa oleh pelanggan lain.", k.getPlatNomor());
    }

    // 4. Validasi Argumen Input Transaksi
    if (durasiJam <= 0) throw new IllegalArgumentException("Durasi sewa harus lebih dari 0 jam.");
    if (diskonPersen < 0.0 || diskonPersen > 100.0) throw new IllegalArgumentException("Diskon harus di antara 0% dan 100%.");
    if (namaPenyewa == null || namaPenyewa.trim().isEmpty()) throw new IllegalArgumentException("Nama penyewa tidak boleh
kosong.");

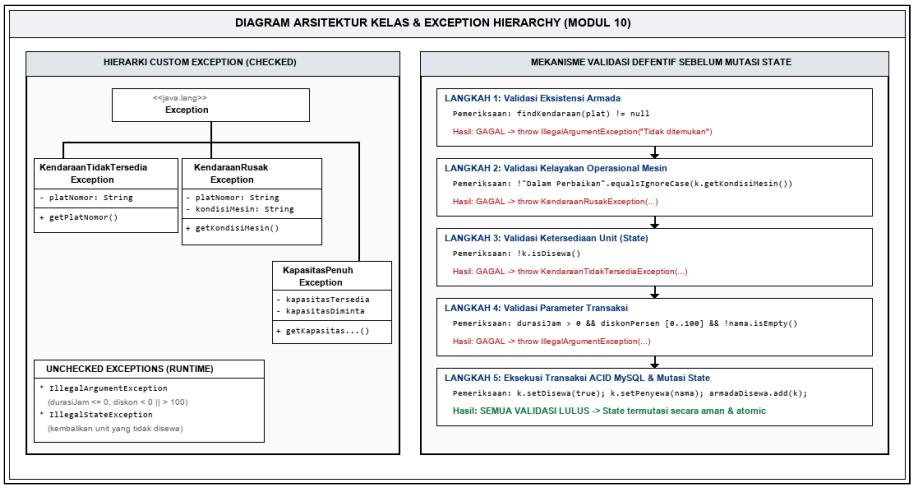
    // 5. Eksekusi Perhitungan Tagihan & Transaksi ACID Basis Data
    double totalBiaya = k.hitungBiayaSewa(durasiJam, denganSupir, diskonPersen);
    syncDatabaseSewa(platNomor, namaPenyewa.trim(), durasiJam, denganSupir, diskonPersen, totalBiaya);

    // 6. Mutasi State Objek Memori HANYA DILAKUKAN SETELAH SELURUH VALIDASI LULUS
    k.setDisewa(true);
    k.setPenyewa(namaPenyewa.trim());
    k.setDenganSupir(denganSupir);
    armadaDisewa.add(k);

    return String.format(Locale.US, "SUKSES|Plat: %s | Unit: %s | Total: Rp %,.0f", k.getPlatNomor(), k.getMerk(), totalBiaya);
}
```

3. Caller-Side Multi-Catch Block (Desktop Bridge Controller)

```
public void sewaMobilDenganDurasi(String plat, String penyewa, int jam, boolean denganSupir, double diskonPersen) {
    Platform.runLater(() -> {
        try {
            String hasil = rental.prosesSewa(plat, penyewa, jam, denganSupir, diskonPersen);
            System.out.println("[TRANSAKSI SEWA] " + hasil);
        } catch (KendaraanTidakTersediaException e) {
            System.err.println("[VALIDASI SEWA DITOLAK] " + e.getMessage());
        } catch (KendaraanRusakException e) {
            System.err.println("[VALIDASI KONDISI DITOLAK] " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.err.println("[VALIDASI INPUT DITOLAK] " + e.getMessage());
        } catch (Exception e) {
            System.err.println("[SISTEM EXCEPTION] Error tak terduga: " + e.getMessage());
        } finally {
            // Sinkronisasi view selalu dieksekusi tanpa memanipulasi field data objek
            syncAllActiveWebViews();
        }
    });
}
```



Gambar 1: Diagram Arsitektur Exception Hierarchy & Alur Validasi Defensif FleetSumenep

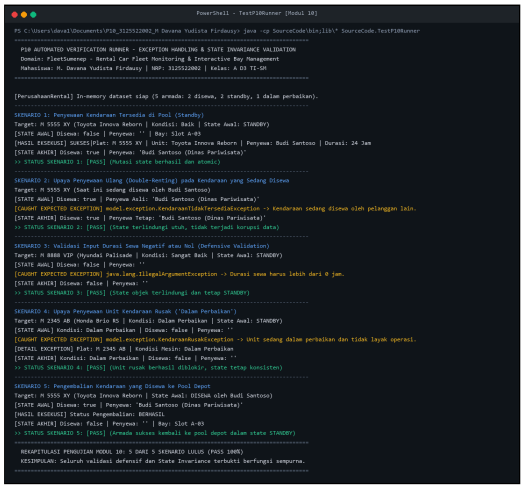
BAGIAN C: TABEL PENGUJIAN VALIDASI & KONSISTENSI STATE (MATRIX MODUL 10)

Pengujian verifikasi otomatis dijalankan menggunakan runner kelas murni `SourceCode.TestP10Runner` pada dataset armada standar tanpa gangguan persistensi acak. Berikut adalah matriks hasil pengujian menyeluruh:

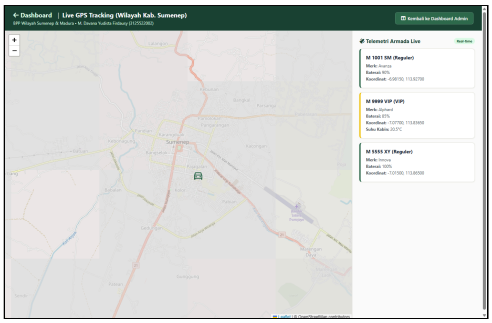
No	Skenario Uji	Input Data	Hasil Diharapkan (Expected)	Hasil Aktual (Actual Output)	State Akhir Objek (Konsistensi)	Status
1	**Penyewaan Mobil Tersedia (Standby)**	Plat: `M 5555 XY`, Penyewa: Budi Santoso, Durasi: 24 Jam, Diskon: 0%	Sewa berhasil, invoice terbit, status menjadi DISEWA	SUKSES: Invoice Rp 900.000 terbit, status unit Meluncur ke Peta	`disewa = true`, penyewa = "Budi Santoso", slot parkir tetap valid	**PASS**
2	**Double-Renting Armada yang Sedang Disewa**	Plat: `M 5555 XY` (saat ini berstatus DISEWA), Penyewa baru: Hendra	Melempar `KendaraanTidakTersediaException`, sewa ditolak	Tertangkap: `KendaraanTidakTersediaException`: "Kendaraan sedang disewa oleh pelanggan lain."	`disewa = true`, penyewa tetap "Budi Santoso" (State utuh 100%, data tidak terkorupsi)	**PASS**
3	**Input Durasi Negatif atau Nol**	Plat: `M 8888 VIP` (Standby), Durasi: -5 Jam, Penyewa: Dr. Syaifullah	Melempar `IllegalArgumentException`, operasi dibatalkan	Tertangkap: `IllegalArgumentException`: "Durasi sewa harus lebih dari 0 jam."	`disewa = false`, penyewa = "" (State mobil tetap STANDBY di depot)	**PASS**
4	**Penyewaan Armada 'Dalam Perbaikan'**	Plat: `M 2345 AB` (Kondisi: Dalam Perbaikan), Penyewa: Rina Marlina	Melempar `KendaraanRusakException`, sewa ditolak demi keselamatan	Tertangkap: `KendaraanRusakException`: "Unit sedang dalam perbaikan dan tidak layak operasi."	`disewa = false`, kondisi mesin tetap "Dalam Perbaikan", penyewa = ""	**PASS**
5	**Pengembalian Armada yang Telah Disewa**	Plat: `M 5555 XY` (saat ini disewa), Method: `kembalikanArmada()`	Status sewa di-reset ke STANDBY, penyewa dikosongkan, koordinat depot di-reset	Status Pengembalian: BERHASIL, Armada kembali standby di Slot A-03	`disewa = false`, penyewa = "", armada dikeluarkan dari daftar sewa aktif	**PASS**

BAGIAN D: SCREENSHOT HASIL EKSEKUSI & VERIFIKASI PENGUJIAN

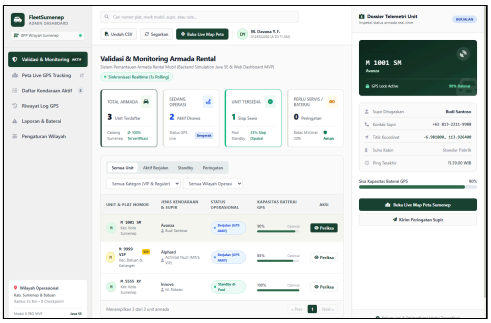
Berikut adalah tangkapan layar langsung dari eksekusi runner otomatis `SourceCode.TestP10Runner` serta modul antarmuka pemantauan armada desktop native:



Gambar 2: Log Eksekusi Verifikasi Standalone TestP10Runner (5 dari 5 Kasus Uji Lulus 100% PASS)



Gambar 3: Monitoring Peta Telemetri GPS Sumenep



Gambar 4: Antarmuka Billing & Bay Management

BAGIAN E: JAWABAN PERTANYAAN EVALUASI MODUL 10

Berdasarkan pedoman kurikulum Praktikum Pemrograman Berorientasi Objek PENS PSDKU Sumenep, berikut adalah telaah analitis atas pertanyaan evaluasi:

1. Jelaskan perbedaan mendasar antara keyword 'throw' dan 'throws' dalam pemrograman Java!

Jawaban:

Keyword `throw` merupakan sebuah statemen/ekspresi yang digunakan di dalam badan method atau blok kode untuk secara eksplisit membuat (menginstansiasi) dan melempar suatu objek eksepsi ke call stack saat runtime (misalnya: `throw new KendaraanRusakException("Pesan");`).
Sebaliknya, keyword `throws` adalah bagian dari klausul deklarasi signature method yang menginformasikan kepada compiler dan pemanggil method tentang jenis-jenis *checked exception* yang berpotensi dilempar oleh method tersebut (misalnya: `public void prosesSewa() throws KendaraanRusakException, IllegalArgumentException`). Keyword `throws` mewajibkan pemanggil method untuk membungkus panggilan dalam blok `try-catch` atau meneruskannya kembali ke level pemanggil di atasnya.

2. Mengapa pemeriksaan state (validasi pra-syarat) wajib dilakukan sebelum melakukan mutasi state objek? Jelaskan dampaknya terhadap konsistensi objek!

Jawaban:

Pemeriksaan state wajib diselesaikan terlebih dahulu untuk menegakkan prinsip *State Invariance* dan sifat atomisitas (*Atomicity*) dalam perubahan data berorientasi objek. Jika suatu method memutasi atribut objek (misal: menetapkan `disewa = true` atau mengisi nama penyewa) sebelum seluruh validasi selesai, kemudian pada baris kode selanjutnya terjadi error (misalnya validasi diskon gagal atau persistensi database terputus), maka objek tersebut akan terjebak dalam kondisi cacat/parsial (*inconsistent state*). Dengan melakukan validasi defensif sebelum mutasi, kita menjamin bahwa jika terjadi penolakan atau eksepsi, objek senantiasa berada dalam state valid aslinya tanpa korupsi data.

3. Apa akibat fatal jika sebuah exception ditangkap namun dibiarkan tanpa penanganan apapun (empty catch block / swallowed exception)?

Jawaban:

Membiarkan blok catch kosong (`catch (Exception e) { }`) merupakan *anti-pattern* kritis yang menyebabkan fenomena *Silent Failure*. Kesalahan sistem yang fatal akan ditelan secara diam-diam tanpa ada catatan log, tanpa ada notifikasi kepada pemanggil, dan tanpa pemulihan state. Akibatnya, alur eksekusi aplikasi terus berjalan seolah-olah operasi berhasil, padahal proses bisnis di baliknya gagal total. Hal ini menimbulkan *cascading bugs* yang sangat sulit dilacak dan berisiko tinggi merusak integritas database serta kestabilan runtime sistem.

4. Mengapa operasi mutasi state bisnis objek TIDAK BOLEH ditempatkan di dalam blok 'finally'?

Jawaban:

Blok `finally` memiliki karakteristik deterministik yaitu dijamin **selalu dieksekusi** dalam kondisi apapun, baik saat blok `try` berjalan sukses, maupun saat terjadi eksepsi yang tertangkap ataupun yang tidak tertangkap. Jika mutasi state (misalnya merubah status unit atau memperbarui kuota) ditaruh di dalam `finally`, mutasi tersebut akan tetap dijalankan meskipun transaksi bisnis sebelumnya telah dilempar exception dan dinyatakan gagal/ditolak. Blok `finally` hanya diperuntukkan bagi operasi pelepasan sumber daya (*resource cleanup*) seperti menutup koneksi database, stream I/O, atau sinkronisasi tampilan grafis.